Taylor-Couette Mixing Research Log 1

Isaiah Bascom

March 10th 2026

1 Introduction

Since last meeting I've worked on:

- Notes and Annotations on 11.1 - 11.3 of Brunton and Kutz [1]
- Mathematical description of Q-learning.
- Breakdown of the Navier-Stokes equation as used in [2]
- Initial Setup of OpenFOAM

2 Takeaways From Brunton and Kutz

2.1 Section 11.1

This was a very helpful description of the basic terms in RL. I particularly enjoyed the derivation of Bellman's equation from the Value function.

$$V(\mathbf{s}) = \max_{\pi} \mathbb{E}\left(\mathbf{r}_0 + \sum_{k=1}^{\infty} \gamma^k \mathbf{r}_k \;\middle|\; \mathbf{s}_1 = \mathbf{s}'\right), \tag{1}$$

$$\implies V(\mathbf{s}) = \max_{\pi} \mathbb{E}(\mathbf{r}_0 + \gamma V(\mathbf{s}')). \tag{2}$$

(See Appendix A)

2.2 Section 11.2 - Model-Based

The discussion of Dynamic Programming was important for my intuition as to why so much of our work in RL is breaking the problem down into subproblems that we can then solve recursively.

Policy Iteration:

1. Fix a policy π(s).
2. Find the corresponding Value function using:

$$V_\pi(\mathbf{s}) = \mathbb{E}(R(\mathbf{s}', \mathbf{s}, \pi(\mathbf{s})) + \gamma V_\pi(\mathbf{s}')) \tag{3}$$

$$= \sum_{s'} P(\mathbf{s}' \mid \mathbf{s}, \pi(\mathbf{s}))(R(\mathbf{s}', \mathbf{s}, \pi(\mathbf{s})) + \gamma V_\pi(\mathbf{s}')). \tag{4}$$

3. Fix our Value function and find the optimal policy.

$$\pi(\mathbf{s}) = \operatorname*{argmax}_{\mathbf{a} \in \mathcal{A}} \mathbb{E}(R(\mathbf{s}', \mathbf{s}, \mathbf{a}) + \gamma V_\pi(\mathbf{s}')) \tag{5}$$

4. Repeat using our new policy until the policy and Value function converge to within a set tolerance.

Assumes a deterministic policy where π(s) = a

Value Iteration:

1. Initialize a random Value function.
2. Update the Value function with

$$V(\mathbf{s}) = \max_{\mathbf{a}} \sum_{s'} P(\mathbf{s}' \mid \mathbf{s}, \mathbf{a})(R(\mathbf{s}', \mathbf{s}, \mathbf{a}) + \gamma V(\mathbf{s}')), \tag{6}$$

   until a certain convergence criterion is met.

3. Once the Value function converges, extract the corresponding policy with

$$\pi(\mathbf{s}, \mathbf{a}) = \operatorname*{argmax}_{\mathbf{a}} \sum_{s'} P(\mathbf{s}' \mid \mathbf{s}, \mathbf{a})(R(\mathbf{s}', \mathbf{s}, \mathbf{a}) + \gamma V(\mathbf{s}')). \tag{7}$$

Notice how the Quality function appears in policy and value iteration.

$$Q(\mathbf{s}, \mathbf{a}) = \mathbb{E}(R(\mathbf{s}', \mathbf{s}, \mathbf{a}) + \gamma V_\pi(\mathbf{s}')) \tag{8}$$

$$= \sum_{s'} P(\mathbf{s}' \mid \mathbf{s}, \mathbf{a})(R(\mathbf{s}', \mathbf{s}, \mathbf{a}) + \gamma V_\pi(\mathbf{s}')). \tag{9}$$

$$\pi(\mathbf{s}, \mathbf{a}) = \operatorname*{argmax}_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a}), \tag{10}$$

$$V(\mathbf{s}) = \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a}). \tag{11}$$

2.3 Section 11.3 - Model-Free

There are two versions of Monte Carlo learning distinguished by their goals. The first is seeking to find the Value function, and the second is seeking to find the Quality function. Two important notes is that Monte Carlo learning operates over an *episode* which is defined as n steps, and we define the cumulative discounted reward as

$$R_\Sigma = \sum_{k=1}^{n} \gamma^k \mathbf{r}_k. \tag{12}$$

1. **Monte Carlo learning of the Value function.**
   For every state s_k visited in the episode we update V(s_k) with

$$V^{\text{new}}(\mathbf{s}_k) = V^{\text{old}}(\mathbf{s}_k) + \frac{1}{n}\left(R_\Sigma - V^{\text{old}}(\mathbf{s}_k)\right) \quad \forall\, k \in [1, \ldots, n]. \tag{13}$$

2. **Monte Carlo learning of the Quality function.**
   For every (s_k, a_k) in the episode update Q(s_k, a_k) with

$$Q^{\text{new}}(\mathbf{s}_k, \mathbf{a}_k) = Q^{\text{old}}(\mathbf{s}_k, \mathbf{a}_k) + \frac{1}{n}\left(R_\Sigma - Q^{\text{old}}(\mathbf{s}_k, \mathbf{a}_k)\right) \quad \forall\, k \in [1, \ldots, n]. \tag{14}$$

Can replace $\frac{1}{n}$ with α as the learning rate.

TD(0)

$$V^{\text{new}}(\mathbf{s}_k) = V^{\text{old}}(\mathbf{s}_k) + \alpha \overbrace{\left(\underbrace{\mathbf{r}_k + \gamma V^{\text{old}}(\mathbf{s}_{k+1})}_{\text{TD target estimates } R_\Sigma} - V^{\text{old}}(\mathbf{s}_k) \right)}^{\text{TD error}} \tag{15}$$

TD(n)

$$R_\Sigma^{(n)} = \mathbf{r}_k + \gamma \mathbf{r}_{k+1} + \gamma^2 \mathbf{r}_{k+2} + \cdots + \gamma^n \mathbf{r}_{k+n} + \gamma^{n+1} V(\mathbf{s}_{k+n+1}) \tag{16}$$

$$= \left(\sum_{j=0}^{n} \gamma^j \mathbf{r}_{k+j} \right) + \gamma^{n+1} V(\mathbf{s}_{k+n+1}). \tag{17}$$

TD(n) converges to the Monte Carlo approach.

Q-Learning

$$Q^{\text{new}}(\mathbf{s}_k, \mathbf{a}_k) = Q^{\text{old}}(\mathbf{s}_k, \mathbf{a}_k) + \alpha \left(r_k + \gamma \max_{\mathbf{a}} Q(\mathbf{s}_{k+1}, \mathbf{a}) - Q^{\text{old}}(\mathbf{s}_k, \mathbf{a}_k) \right). \tag{18}$$

Important to note that this is *off-policy*.

3 Introducing Myself to OpenFOAM

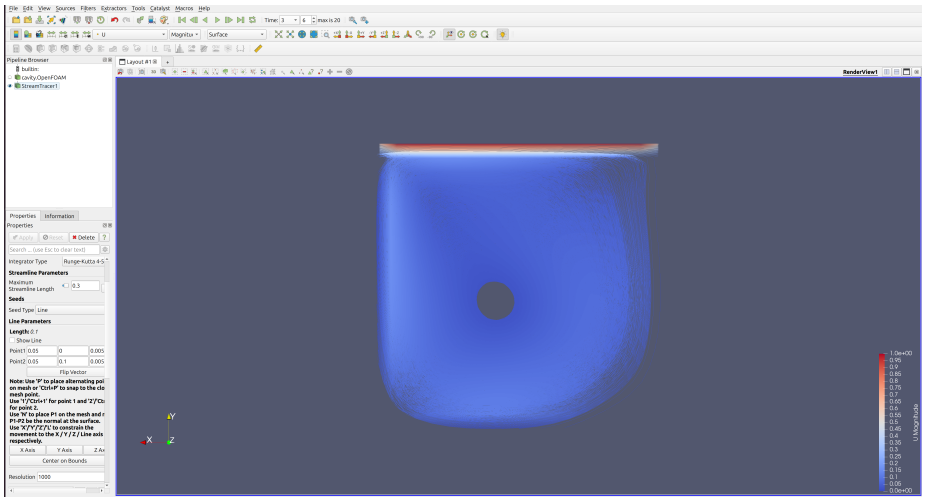

Figure 1: Cavity Example from OpenFOAM in ParaView

3.1 Setup and Example

OpenFOAM was something new for me. I had gone in expecting it to be a standard C++ library, but instead it uses "cases" which are packages of plain text files that are used to define the system.

To dip my toes in the water, I ran through the cavity example [3] OpenFOAM provides that models fluid in a cavity with a moving lid. I utilized OpenFOAM's icoFoam solver which could also be applied to our Taylor-Couette case.

3.2 Takeaways

- We could have our implementation of TD3 simply manipulate the plain text files describing our Taylor-Couette environment and then rerun with the adjusted parameters.
- Our main challenge here will be putting together the specific Taylor-Couette case, but there seem to be plenty of helpful resources online [4].

4 Next Steps

- TD3 reading
  - What is the core iterative process?
  - Describe parameters.
  - Simple preliminary implementation options?
- Develop Taylor-Couette Case.
  - Choices regarding boundary geometry. Preferred dimensions?
- Reward function formulation?
  - How will we define our mixing metric?
  - Is mixing more valuable than energy efficiency?

A Checking (1) $\implies$ (2)

Let

$$V(\mathbf{s}) = \max_{\pi} \left(\mathbb{E} \left(\mathbf{r}_0 + \sum_{k=1}^{\infty} \gamma^k \mathbf{r}_k \;\middle|\; \mathbf{s}_1 = \mathbf{s}' \right) \right).$$

Note that $\mathbb{E}$ is a linear operator. So

$$\begin{aligned} V(\mathbf{s}) &= \max_{\pi} \left(\mathbb{E}\left(\mathbf{r}_0 \mid \mathbf{s}_1 = \mathbf{s}'\right) + \mathbb{E} \left(\sum_{k=1}^{\infty} \gamma^k \mathbf{r}_k \;\middle|\; \mathbf{s}_1 = \mathbf{s}' \right) \right) \\ &= \max_{\pi} \left(\mathbb{E}\left(\mathbf{r}_0 \mid \mathbf{s}_1 = \mathbf{s}'\right) + \mathbb{E} \left(\gamma \sum_{k=1}^{\infty} \gamma^{k-1} \mathbf{r}_k \;\middle|\; \mathbf{s}_1 = \mathbf{s}' \right) \right). \end{aligned}$$

Let $j = k - 1$. Then

$$\begin{aligned} V(\mathbf{s}) &= \max_{\pi} \left(\mathbb{E}\left(\mathbf{r}_0 \mid \mathbf{s}_1 = \mathbf{s}'\right) + \mathbb{E} \left(\gamma \sum_{j=0}^{\infty} \gamma^{j} \mathbf{r}_{j+1} \;\middle|\; \mathbf{s}_1 = \mathbf{s}' \right) \right) \\ &= \max_{\pi} \left(\mathbb{E}\left(\mathbf{r}_0 \mid \mathbf{s}_1 = \mathbf{s}'\right) + \gamma\, \mathbb{E} \left(\sum_{j=0}^{\infty} \gamma^{j} \mathbf{r}_{j+1} \;\middle|\; \mathbf{s}_1 = \mathbf{s}' \right) \right). \end{aligned}$$

By the Markov property, the sequence $\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3, \ldots$ from time 1 conditioned on $\mathbf{s}_1 = \mathbf{s}'$ has the same probability distribution as the sequence $\mathbf{r}_0, \mathbf{r}_1, \mathbf{r}_2, \ldots$ from time 0 conditioned on $\mathbf{s}_0 = \mathbf{s}'$. Therefore,

$$\mathbb{E} \left(\sum_{j=0}^{\infty} \gamma^{j} \mathbf{r}_{j+1} \;\middle|\; \mathbf{s}_1 = \mathbf{s}' \right) = \mathbb{E} \left(\sum_{j=0}^{\infty} \gamma^{j} \mathbf{r}_{j} \;\middle|\; \mathbf{s}_0 = \mathbf{s}' \right).$$

By the principle of optimality, the optimal policy from time 1 onwards achieves $V(\mathbf{s}')$. Taking the expectation over all possible $\mathbf{s}'$ gives

$$V(\mathbf{s}) = \max_{\pi} \left(\mathbb{E} \left(\mathbf{r}_0 + \gamma\, V(\mathbf{s}') \right) \right).$$

This is Bellman's equation. $\square$

References

[1] S. L. Brunton and J. N. Kutz, *Data-Driven Science and Engineering*. https://databookuw.com/databookV2.pdf

[2] T. Lee, J. Kim, and C. Lee, "Turbulence control for drag reduction through deep reinforcement learning," *Physical Review Fluids*, 8(2):024604, 2023.

[3] OpenFOAM Documentation, Lid-driven cavity flow tutorial. https://www.openfoam.com/documentation/tutorial-guide/2-incompressible-flow/2.1-lid-driven-cavity-flow

[4] A. Horvath, Taylor-Couette flow, OpenFOAM Wiki. https://wiki.openfoam.com/Taylor_Couette_flow_by_Andras_Horvath